

UNIVERSITÀ DEGLI STUDI DI UDINE

DIPARTIMENTO DI SCIENZE MATEMATICHE,
INFORMATICHE E FISICHE

IBML - Internet of Things, Big Data, Machine Learning



Internet of Things Progetto Pet Tracker

Autori

Andrea Gioia, 169484
Alessio Nicodemo, 166972
Mattia Nadal, 167372
Francesco Sabot, 167339

169484@spes.uniud.it
166972@spes.uniud.it
167372@spes.uniud.it
167339@spes.uniud.it

Sommario

Il progetto Pet-Tracker propone la realizzazione di un sistema IoT per il tracciamento in tempo reale di animali domestici di media taglia. La soluzione integra un dispositivo embedded basato su ESP32-S3 con connettività LTE e modulo GNSS, in grado di acquisire e trasmettere dati di posizione, stato della batteria e attività motoria verso un backend remoto.

L'architettura segue il modello IoT World Forum (IoTWF) e adotta tecniche di ottimizzazione energetica, tra cui modalità Deep Sleep e strategie di Hot Start per il GPS. I dati vengono elaborati e resi disponibili tramite API REST e visualizzati attraverso un'applicazione mobile dedicata.

L'obiettivo principale è garantire un sistema affidabile, a basso consumo energetico e scalabile per il monitoraggio continuo dell'animale.

Indice

1	Architettura del sistema	4
1.1	Mappatura sul modello IoTWF	4
2	Hardware	4
2.1	LiLyGo T-SIM7670G-S3	4
2.2	Antenna GPS e LTE	4
2.3	Batteria e Power Management	4
2.4	BMA400 CJMCU-400	5
3	Firmware: Logica di funzionamento	5
3.1	Gestione della Persistenza e Hot Start	5
3.2	Protocollo di Comunicazione HTTPS	5
3.3	Risparmio Energetico e Deep Sleep	6
3.4	Monitoraggio Hardware e Batteria	7
4	Back-end e Database	7
4.1	Architettura di Hosting e Network Exposure	7
4.2	Sistema di Backup e Data Retention	7
4.3	Monitoraggio e Business Continuity	8
4.4	Struttura dei dati	8
4.5	Interfaccia API e API Rules	8
4.5.1	API Rules e Sicurezza	8
4.6	Smistamento e Normalizzazione	8
4.6.1	Funzionamento del Trigger onRecordAfterCreateSuccess	9
4.6.2	Macchina a stati delle attività	9
4.6.3	Hook onRecordAfterUpdateSuccess	11
4.6.4	Watchdog di Inattività	11
4.6.5	Vantaggi Architetturali e Corrispondenza IoTWF	11
5	Sistema di Notifiche Push	12
5.1	Analisi dei Vincoli Tecnologici: Incompatibilità Goja e Firebase	12
5.2	Architettura del FCM Bridge	12
5.3	Logica di Invio e Trigger	12
6	Front-end e Applicazione Mobile	13
6.1	Architettura a livelli	13
6.2	Autenticazione e persistenza della sessione	13
6.3	Splash Screen e caricamento parallelo	14
6.4	Comunicazione in tempo reale	14
6.5	Geofencing lato client	14
6.6	Notifiche push	15
6.7	Funzionalità principali	15
7	Test e Risultati	15
8	Limiti del sistema	15

9 Conclusioni	16
Riferimenti bibliografici	17

1 Architettura del sistema

L'architettura del sistema Pet-Tracker segue il **Modello a 7 livelli dell'IoT World Forum (IoTWF)**, garantendo una transizione fluida dal dato fisico all'informazione per l'utente.

1.1 Mappatura sul modello IOTWF

- **Livello 1 (Physical Devices):** Rappresentato dalla board LilyGo e dai sensori (GPS, Accelerometro).
- **Livello 2 (Connectivity):** Gestito tramite il modem LTE e protocolli di trasporto sicuri.
- **Livello 3 (Edge Computing):** Elaborazione locale sul microcontrollore per il filtraggio delle stringhe NMEA e il calcolo dei passi.
- **Livello 4 (Data Accumulation):** Persistenza dei dati sul server PocketBase (database SQLite).
- **Livello 5 (Data Abstraction):** Esposizione dei dati tramite API REST strutturate in formato JSON.
- **Livello 6 (Application):** L'applicazione mobile dedicata che trasforma le coordinate in posizioni su mappa.
- **Livello 7 (Collaboration and Processes):** L'interazione del proprietario con il sistema per il monitoraggio dell'animale.

2 Hardware

2.1 LiLyGo T-SIM7670G-S3

Il dispositivo utilizza una board LiLyGo T-SIM7670G-S3 [1] che si basa sul modulo **ESP32-S3** dual-core, che gestisce sia la logica applicativa che la comunicazione con il modem SIM7670G tramite interfaccia UART.

2.2 Antenna GPS e LTE

Il sistema utilizza un'antenna flessibile LTE [2] per la connettività dati e un'antenna ceramica attiva [3] per il ricevitore GNSS integrato, essenziale per la precisione della localizzazione outdoor.

2.3 Batteria e Power Management

Viene impiegata una batteria ricaricabile 3000mAh 30A Litio-Ion ad alta capacità. Il monitoraggio avviene tramite un partitore di tensione collegato al pin `ADC_BAT` (GPIO 4), permettendo di calcolare la percentuale di carica residua.

2.4 BMA400 CJMCU-400

L'accelerometro usato è un **BMA400** [4] integrato nella breadboard **CJMCU-400**, che viene interrogato per rilevare il movimento, consentendo l'implementazione di un contapassi utile al monitoraggio della salute dell'animale.

3 Firmware: Logica di funzionamento

Il firmware, sviluppato in ambiente **Arduino** per il SoC **ESP32-S3**, gestisce l'acquisizione dei dati dai sensori, il controllo del modem **LTE** e l'ottimizzazione energetica tramite una macchina a stati finiti.

3.1 Gestione della Persistenza e Hot Start

Per ridurre drasticamente il *Time To First Fix* (TTFF) del modulo GNSS, il firmware sfrutta la memoria **RTC (Real-Time Clock)** dell'ESP32. Questa memoria permette di mantenere i dati vitali anche durante i cicli di *Deep Sleep*. Le variabili critiche, come l'ultima posizione valida (`lastLat`, `lastLon`) e i timestamp (`lastGpsDate`, `lastGpsTime`), vengono dichiarate con l'attributo `RTC_DATA_ATTR`.

Al risveglio del dispositivo, la funzione `iniettaGps()` esegue le seguenti operazioni:

- Recupera le coordinate geografiche e temporali salvate nel ciclo precedente.
- Inietta la posizione nota nel modulo GNSS tramite il comando `AT+CGNSSPOS`.
- Sincronizza l'orario del modem con il comando `AT+CGNSSTIME`.

Questo meccanismo abilita la modalità **Hot Start**, che minimizza il tempo di ricerca dei satelliti e, di conseguenza, il consumo energetico.

```
1 void iniettaGps() {  
2   if (!hasGpsFix) return;  
3   // Costruzione comando per Hot Start basato su ultima posizione nota  
4   String cmdPos = "AT+CGNSSPOS=" + formatCoordinate(lastLat, true) + ","  
5     ↪ + latDir + "," +  
6     ↪ formatCoordinate(lastLon, false) + "," + lonDir + "  
7     ↪ ,0,100";  
   sendAT(cmdPos.c_str());  
}
```

Listing 1: Ottimizzazione del fix GPS tramite iniezione di dati persistenti

3.2 Protocollo di Comunicazione HTTPS

La trasmissione dei dati al backend PocketBase è affidata a una serie di comandi AT inviati via seriale al modem SIM7670G. Il firmware incapsula le informazioni in un oggetto JSON strutturato, inviato tramite una richiesta **HTTPS POST**. Il payload include:

- `board_id`: un identificativo univoco derivato dall'IMEI del modem;
- `geo`: un oggetto con latitudine e longitudine per la gestione cartografica;

- **battery / battery_percent / charging:** telemetria sullo stato di carica della batteria;
- **steps:** conteggio dei passi rilevato dall'accelerometro;
- **sleep:** flag booleano che segnala al server l'imminente entrata in stato di inattività;
- **trip:** flag booleano che segnala al server che l'animale si trova su un veicolo in movimento. Quando attivo, ha priorità assoluta sulla logica di geofencing e sopprime il Deep Sleep per garantire la continuità del tracciamento.

```

1 void inviaDati(float l_lat, float l_lon, const BatInfo& bat, const
  ↳ String& timestamp, const StepData& step, bool isSleeping, bool
  ↳ isTrip) {
2   String json = "{";
3   json += "\"timestamp\":\"" + timestamp + "\",";
4   json += "\"geo\":{\"lon\":\"" + String(l_lon, 6) + "\",\"lat\":\"" + String(
  ↳ l_lat, 6) + "\",";
5   json += "\"battery\":\"" + String(bat.voltage, 2) + "\",";
6   json += "\"battery_percent\":\"" + String(bat.percent) + "\",";
7   json += "\"charging\":\"" + String(bat.charging ? "true" : "false") + ",
  ↳ ";
8   json += "\"steps\":\"" + String(step.count) + "\",";
9   json += "\"trip\":\"" + String(isTrip ? "true" : "false") + "\",";
10  json += "\"sleep\":\"" + String(isSleeping ? "true" : "false");
11  json += "}";
12  // Invio tramite comandi AT HTTP
13  modem.print(json);
14 }

```

Listing 2: Esempio di strutturazione del payload JSON

3.3 Risparmio Energetico e Deep Sleep

Il sistema è progettato per massimizzare l'autonomia della batteria da 3000mAh. La logica di risparmio energetico è coordinata con l'accelerometro **BMA400** e tiene conto dello stato di viaggio:

1. **Rilevamento Attività:** Il firmware monitora costantemente il numero di passi. Ogni nuovo movimento resetta un timer di inattività (`lastActivityTime`).
2. **Modalità Viaggio:** Se il flag `trip` è attivo, il Deep Sleep viene soppresso indipendentemente dall'inattività rilevata dall'accelerometro. Il dispositivo continua a trasmettere posizione e telemetria a intervalli regolari per tutto il tempo in cui è in viaggio.
3. **Invio Finale:** Se non viene rilevata attività per un tempo superiore a `SLEEP_TIMEOUT` (30 secondi) e il flag `trip` non è attivo, il sistema invia un ultimo pacchetto dati aggiornando il flag `sleep` a `true`.
4. **Ibernazione:** Il microcontrollore entra in *Deep Sleep*, spegnendo i moduli radio. Il risveglio è gestito tramite interrupt hardware sul pin `GPIO_5`, generato dal BMA400 solo al rilevamento di un nuovo movimento.

```

1 void enterDeepSleep() {
2   Serial.println("\nEntro in Deep Sleep...");

```

```

3 // Configura il pin di risveglio (GPIO 5) collegato al BMA400
4 pinMode(WAKEUP_PIN, INPUT_PULLDOWN);
5 esp_sleep_enable_ext0_wakeup(WAKEUP_PIN, 1);
6 esp_deep_sleep_start();
7 }

```

Listing 3: Configurazione del Deep Sleep con risveglio tramite accelerometro

3.4 Monitoraggio Hardware e Batteria

Il firmware gestisce in modo intelligente l'alimentazione per evitare letture errate o consumi parassiti:

- **Controllo ADC:** Il circuito del partitore di tensione per la batteria viene attivato solo durante la lettura tramite il pin `BAT_ADC_EN`, garantendo la precisione del voltaggio acquisito dal pin `ADC_BAT`.
- **Stato USB:** Attraverso la lettura dei registri hardware dell'ESP32, il sistema rileva se il dispositivo è collegato a una fonte di ricarica USB, settando dinamicamente il campo `charging` nel pacchetto dati.

4 Back-end e Database

Il centro nevralgico della gestione dati è basato su **PocketBase** [5], una soluzione open-source che integra un database embedded SQLite con un'interfaccia API REST.

4.1 Architettura di Hosting e Network Exposure

L'istanza di PocketBase è stata ospitata su una workstation locale e data la natura del sistema IoT, che richiede la raggiungibilità del backend da parte del dispositivo (tramite rete LTE) e dell'applicazione mobile (tramite rete dati/Wi-Fi), si è reso necessario esporre il servizio locale su rete pubblica. Il problema del *NAT Traversal* e dell'assenza di un indirizzo IP pubblico statico sulla rete locale è stato risolto mediante l'impiego di **ngrok** [6]. Questa tecnologia di *tunneling* crea un tunnel sicuro (HTTPS) tra l'endpoint pubblico di ngrok e la porta locale su cui è in esecuzione il server. Tale URL funge da *Reverse Proxy*, reindirizzando il traffico HTTPS in ingresso verso l'istanza locale di PocketBase, garantendo così la persistenza dei dati e l'accessibilità dei servizi in tempo reale ovunque sia presente una connessione internet.

4.2 Sistema di Backup e Data Retention

Per garantire la resilienza dei dati e prevenire perdite accidentali dovute a guasti del server locale, è stato implementato un sistema di backup automatizzato. Il processo utilizza l'utility **rclone** [7], uno strumento a riga di comando per la gestione del cloud storage, configurato per eseguire la sincronizzazione della directory dei dati di PocketBase verso un bucket remoto su **Backblaze B2** [8]. L'automazione è gestita tramite un *cron job* che esegue il backup in modo incrementale ogni primo giorno del mese.

4.3 Monitoraggio e Business Continuity

Per garantire l'affidabilità dell'intero ecosistema, è stato implementato un sistema di monitoraggio esterno tramite **UptimeRobot** [9]. Dato che l'accessibilità del backend dipende dalla stabilità del tunnel ngrok, il monitoraggio interroga l'endpoint HTTPS a intervalli regolari, inviando notifiche immediate in caso di downtime.

4.4 Struttura dei dati

I dati ricevuti vengono normalizzati e memorizzati in una collezione denominata `data_sent_raw`. La struttura del record prevede i seguenti campi:

- `timestamp`: riferimento temporale dell'acquisizione.
- `lat / lon`: coordinate geografiche acquisite dal modulo GNSS;
- `battery`: stato di carica della batteria (espresso in voltaggio);
- `battery_percent`: stato di carica della batteria (espresso in percentuale);
- `charging`: booleano che indica se la batteria è in corso di ricarica;
- `steps`: conteggio dei passi rilevato dall'accelerometro BMA400;
- `sleep`: booleano che indica se il dispositivo è in stato di inattività prolungata;
- `trip`: booleano che indica se l'animale si trova su un veicolo in movimento.

4.5 Interfaccia API e API Rules

L'accesso ai dati memorizzati nel backend avviene esclusivamente tramite un'interfaccia **RESTful API** fornita nativamente da PocketBase. Questo approccio permette di separare nettamente il livello di persistenza (Database) dal livello di fruizione (Applicazione Mobile).

4.5.1 API Rules e Sicurezza

Per garantire l'integrità e la riservatezza dei dati, sono state configurate specifiche **API Rules** basate su espressioni booleane:

- **List/View**: Visualizzazione permessa solo agli utenti autenticati.
- **Create**: Abilitata per il dispositivo LilyGo tramite autenticazione dedicata.
- **Update/Delete**: Ristrette ai soli amministratori del sistema.

4.6 Logica di Smistamento e Normalizzazione (Server-side Hooks)

Per ottimizzare la gestione dei dati e garantire la separazione delle responsabilità, è stata implementata una logica di smistamento automatico lato server tramite i *JavaScript Event Hooks* di PocketBase. Questa architettura permette di mantenere la collezione `data_sent_raw` come un punto di accumulo "grezzo", delegando al server il compito di distribuire le informazioni nelle tabelle specializzate.

4.6.1 Funzionamento del Trigger `onRecordAfterCreateSuccess`

Il sistema sfrutta l'hook `onRecordAfterCreateSuccess`, che si attiva immediatamente dopo il salvataggio di un nuovo record inviato dalla board LilyGo. Il processo opera su tre direttrici principali:

- **Persistenza Geografica ed Energetica:** I parametri di tensione, percentuale di carica e coordinate GPS vengono estratti e duplicati nelle collezioni `battery_data` e `positions_duplicate`. L'uso di blocchi *try-catch* indipendenti garantisce che un errore in un sensore secondario non comprometta il tracciamento GPS.
- **Logica di Sessione (Activity Tracking):** Il trigger implementa una macchina a stati che tiene conto dei parametri `sleep`, `trip` e della posizione GPS rispetto alle geofence configurate. Se il dispositivo è attivo, il sistema verifica l'esistenza di una sessione: in caso positivo aggiorna il conteggio passi e lo stato, in caso negativo crea una nuova attività.
- **Gestione dello Stato Sleep:** Al ricevimento di un pacchetto con `sleep: true`, il comportamento dipende dallo stato corrente: se l'animale era in viaggio (`status = "v"`), la sessione rimane aperta con stato `"a"` (trip-sleep) per continuare a ricevere posizioni GPS; altrimenti la sessione viene chiusa normalmente con `is_active = false`.
- **Deduplicazione delle Sessioni:** Per evitare la frammentazione delle sessioni in caso di brevi interruzioni del segnale, il sistema verifica se esiste una sessione chiusa di recente (entro 120 secondi) con uno stato compatibile, riattivandola invece di crearne una nuova.

4.6.2 Macchina a stati delle attività

Il cuore della logica server-side è una macchina a stati che modella il comportamento dell'animale. Gli stati sono distinti in *attivi* (dispositivo sveglio) e *sleep* (dispositivo in Deep Sleep), con una corrispondenza biunivoca tra le due categorie.

Stato attivo	Stato sleep	Descrizione
i — inside	d	Animale all'interno della geofence
s — search	p	Ricerca attiva: animale fuori zona con allarme
f — found	h	Animale localizzato fuori zona (senza allarme)
w — walk	z	Animale in passeggiata fuori zona
v — vehicle/trip	a	Animale su veicolo (sessione non chiusa durante sleep)
n — no geofence	n	Nessuna geofence configurata

Tabella 1: Macchina a stati delle attività: corrispondenza tra stati attivi e sleep

La transizione $v \rightarrow a$ è l'unico caso in cui il passaggio in Deep Sleep *non* chiude la sessione: la sessione rimane `is_active = true` con status `"a"`, permettendo di collegare

correttamente le posizioni GPS successive. Al risveglio, se `trip` è ancora attivo, lo stato torna a "v" senza aprire una nuova sessione.

Le transizioni tra stati attivi sono governate dalla funzione `checkGeofences` che, ricevuto un pacchetto, calcola il nuovo stato in base alla posizione rispetto ai poligoni configurati, alla presenza di un allarme utente e all'eventuale flag `trip`. La priorità di valutazione è:

1. Se `trip = true` $\Rightarrow$ stato "v" (priorità assoluta, la geofence non viene calcolata).
2. Se il dispositivo era in "v" e `trip` torna `false` $\Rightarrow$ ricalcolo pulito della geofence senza stato precedente.
3. Altrimenti $\Rightarrow$ la macchina a stati valuta la posizione rispetto alle geofence e l'eventuale allarme utente per determinare la transizione corretta.

```
1 onRecordAfterCreateSuccess((e) => {
2   const utils = require(`${__hooks}/utils.js`);
3   const raw = e.record;
4   const boardId = raw.getString("board_id");
5   const sleep = raw.getBool("sleep");
6   const trip = raw.getBool("trip");
7   const steps = raw.getInt("steps");
8   const lat = raw.getFloat("lat");
9   const lon = raw.getFloat("lon");
10  const hasCoords = !(lat === 0.0 && lon === 0.0);
11
12  // Recupera eventuale sessione attiva
13  let activeActivity = null;
14  const active = e.app.findRecordsByFilter(
15    "activities", "board_id = {id} && is_active = true",
16    "-id", 1, 0, { id: boardId }
17  );
18  if (active.length > 0) activeActivity = active[0];
19
20  if (!sleep) {
21    const prevStatus = activeActivity
22      ? activeActivity.getString("status") : null;
23
24    // trip ha priorità assoluta sulla geofence
25    const status = (hasCoords || trip)
26      ? utils.checkGeofences(e.app, boardId, lat, lon, prevStatus,
27        ↪ trip)
28      : prevStatus ?? "n";
29
30    if (activeActivity) {
31      if (status === prevStatus) {
32        // Stesso stato: aggiorna passi e fine sessione
33        activeActivity.set("total_steps",
34          activeActivity.getInt("total_steps") + steps);
35        activeActivity.set("end_time", timestamp);
36        e.app.save(activeActivity);
37      } else {
38        // Cambio di stato: chiudi e apri nuova sessione
39        activeActivity.set("end_time", timestamp);
40        activeActivity.set("is_active", false);
41        e.app.save(activeActivity);
42        // ... crea nuovo record activities con nuovo status
43      }
44    }
45  }
46}
```

```

43     }
44   } else if (activeActivity) {
45     const prevStatus = activeActivity.getString("status");
46     if (prevStatus === "v") {
47       // Sleep durante viaggio: non chiude la sessione,
48       // cambia solo lo status in "a" (trip-sleep)
49       activeActivity.set("status", "a");
50       activeActivity.set("end_time", timestamp);
51       e.app.save(activeActivity);
52     } else {
53       // Sleep normale: chiude la sessione
54       activeActivity.set("is_active", false);
55       activeActivity.set("end_time", timestamp);
56       e.app.save(activeActivity);
57     }
58   }
59   e.next();
60 }, "data_sent_raw");

```

Listing 4: Implementazione della logica di sessione con macchina a stati completa

4.6.3 Hook onRecordAfterUpdateSuccess

Oltre all'hook di creazione, è presente un secondo hook `onRecordAfterUpdateSuccess` registrato sulla collezione `activities`. Il suo scopo è gestire le transizioni di stato che avvengono per aggiornamento del record piuttosto che per creazione, in particolare:

- "v" → "a": l'animale entra in sleep mentre è ancora in viaggio. La sessione non viene chiusa, ma lo status viene aggiornato a "a" e viene salvato un evento `status_trip_sleep`.
- "a" → "v": l'animale si risveglia e il flag `trip` è ancora attivo. La sessione esistente viene mantenuta e lo status torna a "v", salvando un evento `status_trip`.

Questo secondo hook è necessario perché `onRecordAfterCreateSuccess` intercetta solo i record *nuovi* nella collezione `activities`, mentre queste transizioni avvengono su record già esistenti tramite `app.save()`.

4.6.4 Watchdog di Inattività

Un *cron job* eseguito ogni minuto controlla che nessuna sessione rimanga aperta indefinitamente a causa di un'interruzione imprevista del dispositivo (perdita di segnale LTE, spegnimento improvviso). Se una sessione attiva non riceve aggiornamenti per più di 10 minuti, viene chiusa automaticamente impostando `is_active = false` e il flag `anomaly = true`, e viene salvato un evento `watchdog` per tracciabilità.

4.6.5 Vantaggi Architetture e Corrispondenza IoTWF

Questa implementazione riflette i principi del Modello IoTWF in due punti chiave:

1. **Efficienza del Livello 3 (Edge):** La LilyGo invia un unico pacchetto "bulk", riducendo il tempo di attività del modem LTE e preservando la batteria.

2. **Ottimizzazione del Livello 5 (Data Abstraction):** L'applicazione Flutter non deve processare dati grezzi; interroga API già normalizzate dal trigger, garantendo una visualizzazione fluida della cronologia.

5 Sistema di Notifiche Push

L'implementazione di un sistema di notifiche push è fondamentale per un dispositivo di tracciamento animali, in quanto permette di avvisare tempestivamente l'utente in caso di pericoli (uscita dal perimetro sicuro) o problemi tecnici (batteria scarica).

5.1 Analisi dei Vincoli Tecnologici: Incompatibilità Goja e Firebase

Durante la fase di sviluppo, è emerso un vincolo tecnico significativo riguardante l'integrazione tra il backend PocketBase e l'SDK ufficiale di Firebase. PocketBase è scritto in linguaggio Go e utilizza **Goja** come motore di esecuzione JavaScript per la gestione della logica server-side (hooks). Goja è un'implementazione di ECMAScript 5.1 che presenta limitazioni strutturali rispetto a un ambiente Node.js [10] standard:

- **Mancanza di moduli nativi:** Goja non supporta i moduli core di Node.js (come `fs`, `crypto` o `net`), indispensabili per il funzionamento dell'SDK di Firebase.
- **Runtime isolato:** L'SDK `firebase-admin` [11] richiede un ambiente di esecuzione completo per gestire l'autenticazione tramite Service Account e le connessioni persistenti ai server Google.

Per superare queste limitazioni, è stata adottata un'architettura a **Bridge**, separando la logica di business dall'invio fisico dei messaggi.

5.2 Architettura del FCM Bridge

Il sistema si basa su un microservizio esterno sviluppato in Node.js [10], denominato *FCM Bridge*. L'interazione avviene secondo il seguente flusso:

1. **PocketBase (Goja):** Rileva un evento (es. batteria scarica) e invia una richiesta HTTP POST al bridge tramite il modulo interno `$http.send`.
2. **FCM Bridge (Node.js):** Riceve la richiesta, carica il Service Account di Firebase e inoltra la notifica tramite l'SDK ufficiale [11].
3. **Feedback e pulizia token:** Il bridge restituisce lo stato della consegna. In caso di errore 404 (token non registrato, app disinstallata) o 400 (token malformato), il backend provvede alla rimozione automatica del token dal profilo utente. I token FCM sono memorizzati come array JSON per supportare più dispositivi per singolo utente.

5.3 Logica di Invio e Trigger

Le notifiche vengono scatenate dai seguenti eventi, ciascuno con logica contestuale per evitare invii ridondanti:

Evento	Condizione di trigger	Nota
Uscita dalla zona	$i \rightarrow s$ oppure $i \rightarrow w$	Dipende dalla presenza di allarme utente
Rientro autonomo	$s \rightarrow i$	
Animale trovato	$s \rightarrow f$	Allarme disattivato
Ritorno in zona	$f \rightarrow i$	
Allarme da passeggiata	$w \rightarrow s$	Allarme riattivato
Inizio viaggio	qualsiasi $\rightarrow v$	Solo al primo pacchetto trip
Fine viaggio	$v \rightarrow$ qualsiasi	Ricalcolo geofence
Batteria bassa	$\leq 20\%$ (cambio soglia)	Solo al cambio di fascia
Batteria critica	$\leq 10\%$ (cambio soglia)	Solo al cambio di fascia
Batteria in carica	charging passa a true	Solo al cambio di stato
Watchdog	Silenzio > 10 minuti	Chiusura automatica sessione

Tabella 2: Trigger del sistema di notifiche push

6 Front-end e Applicazione Mobile

L'applicazione mobile è stata sviluppata in Flutter [12], un framework cross-platform che permette di produrre un'unica codebase per Android e iOS. La comunicazione con il backend avviene tramite il PocketBase SDK ufficiale per Dart, che gestisce internamente le chiamate REST e i meccanismi di autenticazione.

6.1 Architettura a livelli

Il codice dell'applicazione è organizzato secondo una separazione netta tra livelli di responsabilità:

- **Repositories:** accesso ai dati remoti tramite PocketBase SDK, con esposizione di metodi asincroni e stream tipizzati verso le schermate.
- **Models:** rappresentazione del dominio applicativo (es. Utente, Posizione, Attività). Questo livello si occupa della deserializzazione dei dati, mappando i record grezzi (JSON) ricevuti dal backend in oggetti Dart fortemente tipizzati.
- **Services:** logica trasversale e di sistema, indipendente dalla UI (autenticazione, logica di geofencing, tracciamento GPS, notifiche push).
- **Screens:** interfaccia utente che si limita a richiedere i dati ai repository e a reagire ai loro aggiornamenti.

Questa architettura facilita la testabilità e disaccoppia la logica di business dalla presentazione, in linea con il principio di separazione delle responsabilità.

6.2 Autenticazione e persistenza della sessione

L'autenticazione è gestita tramite un `SecureAuthStore` personalizzato che estende l'`AuthStore` nativo di PocketBase. Il token JWT restituito dal server al momento del login viene se-

rializzato in JSON e salvato in modo cifrato tramite `flutter_secure_storage`, che su Android si appoggia al sistema Keystore e su iOS al Keychain.

Al riavvio dell'app, il token viene riletto dallo storage sicuro e viene tentato un refresh automatico della sessione. In caso di successo, l'utente accede direttamente alla schermata principale senza dover reinserire le credenziali; in caso di token scaduto, lo store viene svuotato e l'utente viene reindirizzato al login.

6.3 Splash Screen e caricamento parallelo

La schermata di avvio svolge un ruolo attivo nel precaricare i dati necessari alla home page. Una volta verificata l'autenticazione, vengono avviate in parallelo, tramite `Future.wait`, le seguenti operazioni:

- recupero del profilo utente e dello stato dell'allarme;
- recupero dell'ultima posizione GPS registrata dal dispositivo;
- recupero delle attività del giorno corrente filtrate per `board_id`.

Il `board_id`, identificativo univoco della board LilyGo associata all'utente, viene risolto dinamicamente interrogando la collezione `boards` sul backend, evitando qualsiasi valore hardcoded. Contestualmente, viene calcolata la zona di geofencing in cui si trova l'animale. I dati precaricati vengono poi passati direttamente alla schermata principale, eliminando ulteriori chiamate di rete al momento della navigazione.

6.4 Comunicazione in tempo reale

Per le funzionalità che richiedono aggiornamenti continui — posizione GPS e stato della batteria — l'app sfrutta il meccanismo di *subscribe* offerto da PocketBase, basato su Server-Sent Events (SSE). I repository espongono uno `Stream` tipizzato (rispettivamente `Stream<Positions>` e `Stream<BatteryData>`) a cui le schermate si sottoscrivono tramite `StreamSubscription`. Ogni nuovo record inserito nel backend viene notificato istantaneamente all'app senza necessità di polling.

6.5 Geofencing lato client

La logica di geofencing è implementata interamente lato client tramite l'algoritmo *Ray-Casting*. Dato un punto GPS (latitudine, longitudine) e un poligono definito da una lista di vertici, l'algoritmo traccia un raggio orizzontale dal punto verso l'infinito e conta le intersezioni con i lati del poligono: un numero dispari indica che il punto è interno al poligono.

Le zone vengono recuperate dalla collezione `geofences` su PocketBase, dove ogni record memorizza il nome della zona, i dati dell'indirizzo e la lista dei vertici del poligono. Solo le zone con il flag `is_active` impostato a `true` vengono considerate nel calcolo. Il risultato della verifica è una stringa con il nome della zona di appartenenza, oppure *“Fuori zona sicura”* se il punto non rientra in nessuna area attiva. Questa informazione viene mostrata in tempo reale nella schermata principale.

6.6 Notifiche push

Le notifiche push sono gestite tramite Firebase Cloud Messaging (FCM). Al primo avvio, l'app mostra un dialog personalizzato che spiega all'utente il motivo della richiesta del permesso, prima di invocare la richiesta di sistema. Una volta ottenuto il consenso, il token FCM del dispositivo viene sincronizzato con il profilo utente su PocketBase: il campo `tokenFCM` è modellato come una lista, permettendo all'utente di ricevere notifiche su più dispositivi contemporaneamente. Un controllo di duplicazione evita che lo stesso token venga registrato più volte.

6.7 Funzionalità principali

Mappa in tempo reale. La schermata principale mostra la posizione aggiornata dell'animale su mappa tramite la libreria `flutter_map` con tile OpenStreetMap. La posizione viene aggiornata ad ogni evento ricevuto dallo stream in tempo reale. L'indirizzo corrispondente alle coordinate viene risolto in forma testuale tramite il servizio di reverse geocoding Nominatim.

Riepilogo giornaliero. La schermata di recap, accessibile selezionando una data dal calendario, recupera dal backend le attività e le posizioni GPS relative a quel giorno. Le posizioni vengono rappresentate come una polilinea sulla mappa, restituendo una visualizzazione del percorso effettuato dall'animale. Le statistiche aggregate (passi totali, distanza stimata in chilometri e durata complessiva del movimento) vengono calcolate client-side a partire dai dati ricevuti.

Stato della batteria. La schermata dedicata mostra la percentuale di carica residua tramite un indicatore circolare animato con gradiente. Quando il campo `charging` del record è impostato a `true`, l'indicatore entra in modalità animazione rotante per segnalare visivamente la ricarica in corso. La durata stimata residua viene calcolata proporzionalmente alla capacità nominale della batteria (3000 mAh).

Gestione delle zone sicure. L'app consente di creare, modificare e disattivare le zone di geofencing tramite un editor poligonale interattivo su mappa. L'utente può disegnare il perimetro della zona toccando i vertici direttamente sulla mappa; al salvataggio, le coordinate dei vertici e il centroide del poligono vengono persistiti su PocketBase. L'indirizzo associato alla zona viene ricavato automaticamente tramite geocoding inverso a partire dal centroide.

7 Test e Risultati

8 Limiti del sistema

Nonostante i risultati ottenuti, il sistema presenta alcune limitazioni:

- **Dipendenza dalla rete LTE:** in assenza di copertura, la trasmissione dei dati non è garantita.
- **Prestazioni indoor:** il modulo GNSS risulta poco affidabile in ambienti chiusi.

- **Backend locale:** l'utilizzo di PocketBase su macchina locale, esposto tramite ngrok, introduce dipendenze da servizi esterni e limita la scalabilità.
- **Consumo energetico del modem LTE:** la trasmissione dati rappresenta la principale fonte di consumo.

9 Conclusioni

Riferimenti bibliografici

- [1] *T-SIM7670G-S3 Specifications*. URL: <https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/docs/en/esp32s3/sim7670g-s3/REAMDE.MD>.
- [2] *LTE Antenna Specifications*. URL: <https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/datasheet/LTE%20Antenna%20Specifications.pdf>.
- [3] *GPS Antenna Specifications*. URL: <https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/datasheet/GPS%20Antenna%20Specifications.pdf>.
- [4] *BMA400 CJMCU-400 Datasheet*. URL: https://www.bosch-sensortec.com/media/boschsensortec/downloads/product_flyer/bst-bma400-fl000.pdf.
- [5] *PocketBase Documentation*. URL: <https://pocketbase.io/docs/>.
- [6] *ngrok Documentation - HTTP Tunnels*. URL: <https://ngrok.com/docs/secure-tunnels/>.
- [7] *rclone - rsync for cloud storage*. URL: <https://rclone.org/>.
- [8] *Backblaze B2 Cloud Storage*. URL: <https://www.backblaze.com/b2/cloud-storage.html>.
- [9] *UptimeRobot - Free Website Monitoring Service*. URL: <https://uptimerobot.com/>.
- [10] *Node.js Documentation*. URL: <https://nodejs.org/en/docs/>.
- [11] *Firebase Admin SDK - Cloud Messaging*. URL: <https://firebase.google.com/docs/cloud-messaging/server>.
- [12] *Flutter Documentation*. URL: <https://docs.flutter.dev/>.